

PZaaS

Pizza as a Service

DOCUMENTAÇÃO TÉCNICA FINAL

Serviço 01 - API Gateway

Integrantes: Davis e Vitor Hugo

Disciplina: Arquitetura de Serviços em Nuvem

Versão da API: v1

Data de consolidação: 16/09/2026

Base URL do Gateway

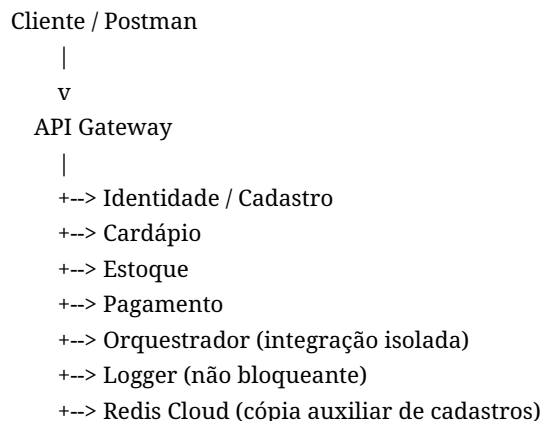
<https://veagabronx2k.app.n8n.cloud/webhook>

1. Visão geral

O API Gateway é o ponto de entrada do PZaaS. A implementação centraliza o acesso aos serviços integrados, valida a chave global da turma, gera e propaga um identificador de correlação (x-pedido-id), aplica rate limit no checkout e padroniza respostas de sucesso e erro.

O fluxo de checkout respeita o estado retornado pelos serviços externos. Em particular, a disponibilidade das pizzas não é alterada pelo Gateway: se o Cardápio informar uma pizza como indisponível, o checkout é interrompido com erro 400.

2. Arquitetura implementada



No checkout, o Pagamento é responsável por acionar o Orquestrador conforme o contrato de integração utilizado. Por isso, o Gateway não chama novamente o Orquestrador depois do Pagamento, evitando criação duplicada de pedidos.

3. Contrato global

Item	Implementação
Versão	v1
Formato	HTTP/JSON
Chave global	Header x-api-key
Valor utilizado	turma2026
Correlação	x-pedido-id gerado pelo Gateway
Health check	GET /health
Rate limit do checkout	5 requisições por 60 segundos
Retry	até 3 tentativas totais em integrações configuradas
Fallback A -> B	pendente onde a URL oficial B não foi fornecida

4. Segurança e rastreabilidade

4.1 x-api-key

As rotas do Gateway validam o header x-api-key antes de encaminhar a chamada aos serviços. Quando ausente ou inválido, a resposta padronizada é HTTP 401.

```
{"error":"UNAUTHORIZED","message":"Chave de API ausente ou inválida."}
```

4.2 x-pedido-id

No checkout, o Gateway gera um pedido_id_gateway no início do fluxo e utiliza o mesmo valor como x-pedido-id nas chamadas para Identidade, Cardápio, consulta de Estoque, Pagamento, baixa de Estoque e Logger. Isso permite correlacionar as etapas de uma mesma requisição.

4.3 Rate limit

O checkout possui limite de 5 requisições por janela de 60 segundos. O teste em produção foi realizado com seis chamadas consecutivas: as cinco primeiras seguiram o fluxo normal e a sexta retornou HTTP 429.

```
{"error":"RATE_LIMIT","message":"Limite de requisições excedido."}
```

5. Endpoints do API Gateway

Método	Rota	Responsabilidade	Situação
GET	/health	Saúde do Gateway	Implementado
GET	/v1/menu	Proxy do Cardápio	Implementado/testado
GET	/v1/estoque/consulta	Consulta de ingrediente	Implementado/testado
POST	/v1/pagamento	Integração isolada de Pagamento	Implementado/testado
POST	/v1/pedido	Integração isolada com Orquestrador	Implementado/testado
POST	/v1/auth/validar	Validação de token	Implementado/testado
POST	/v1/clientes	Criação de cliente + Redis auxiliar	Implementado/testado
POST	/v1/checkout	Fluxo principal de compra	Implementado/testado até regra real do Cardápio
GET	/v1/metrics	Métricas/configuração operacional	Implementado/testado

6. GET /health

Endpoint obrigatório para demonstrar que o Gateway está online.

```
GET https://veagabronx2k.app.n8n.cloud/webhook/health
```

Resultado: implementado e utilizado como health check do Serviço 01.

7. GET /v1/menu - Cardápio

O Gateway encaminha a consulta para o endpoint oficial do Cardápio. O serviço retorna pizzas, ingredientes, preços, disponibilidade e promoções. O Gateway não modifica o campo disponível.

```
GET https://veagabronx2k.app.n8n.cloud/webhook/v1/menu
Header: x-api-key: turma2026
```

Tratamentos configurados/testados: 200, 401, 404 e erro 5xx.

O retry está configurado no fluxo. A versão B não foi documentada com URL oficial nos materiais disponíveis, portanto não foi inventado um fallback.

8. GET /v1/estoque/consulta - Estoque

```
GET https://veagabronx2k.app.n8n.cloud/webhook/v1/estoque/consulta?ingrediente=mussarela
Header: x-api-key: turma2026
```

O Gateway consulta a quantidade disponível de um ingrediente. Resposta real observada para consultas válidas: objeto com status SUCCESS e quantidade.

```
{"status":"SUCCESS","quantidade":0}
```

Foram configurados tratamentos para 400, 401 e 500. Durante os testes, um ingrediente inexistente produziu 500 no serviço externo, embora a documentação do Estoque descreva 400; essa divergência é registrada como comportamento observado.

9. POST /v1/pagamento - Pagamento

Rota de integração isolada para o serviço de Pagamento. O Gateway envia cliente_id, token, valor, metodo_pagamento e itens. Foram observados/testados sucesso 200 e erro 400; também foram configurados 401, 403, 422, 503 e timeout.

Não foi aplicado retry cego ao Pagamento, pois uma repetição após débito sem confirmação pode duplicar cobrança. Em eventual fallback/retry, o mesmo x-pedido-id deve ser preservado para permitir idempotência.

10. POST /v1/pedido - Orquestrador

Integração isolada do Gateway com o Orquestrador. O teste direto retornou HTTP 201 e um pedido com status preparando.

```
POST https://veagabronx2k.app.n8n.cloud/webhook/v1/pedido
{
  "cliente_id": "cli_99",
  "itens": [{"pizza":"calabresa","quantidade":1}]
}
```

Esta rota permanece válida como demonstração de integração, mas não é chamada novamente pelo Gateway após o Pagamento no checkout, pois o próprio Pagamento notifica o Orquestrador.

11. POST /v1/auth/validar - Identidade

Valida o token do cliente no serviço oficial de Cadastro/Identidade. Durante a depuração, o Body foi alterado para JSON explícito para garantir que o token fosse serializado corretamente.

POST /v1/auth/validar

Headers:

x-api-key: turma2026

Content-Type: application/json

x-pedido-id: <id gerado pelo Gateway>

Body:

```
{"token": "<token do cliente>"}
```

Teste final: um token recém-gerado retornou 200 diretamente no serviço e passou corretamente pela validação dentro do checkout após a correção do Body.

12. POST /v1/clientes - Cadastro + Redis auxiliar

Foi adicionada uma rota no Gateway para encaminhar a criação de cliente ao Cadastro oficial. Após sucesso, uma cópia auxiliar é gravada no Redis Cloud. O Redis não substitui a persistência do serviço de Cadastro e é utilizado apenas como recurso auxiliar/demonstrativo do Gateway.

POST https://veagabronx2k.app.n8n.cloud/webhook/v1/clientes

Headers:

x-api-key: turma2026

Content-Type: application/json

Body:

```
{
  "nome": "Cliente Exemplo",
  "email": "cliente.exemplo@pzaas.dev",
  "senha": "<senha>",
  "saldoInicial": 250
}
```

Resposta real de produção observada: HTTP 201 Created.

```
{
  "id": 19,
  "nome": "Cliente Redis Producao3",
  "email": "cliente.redis.producao03@pzaas.dev",
  "saldo": 250,
  "criadoEm": "2026-09-15T12:43:55.301-03:00",
  "message": "Cliente cadastrado com sucesso"
}
```

A chave Redis segue o padrão cliente:<id>. O Value contém id, nome, email, saldo e criadoEm. Senha e token não são armazenados na cópia Redis. Um Redis GET para cliente:16 recuperou com sucesso o JSON previamente salvo.

13. POST /v1/checkout - Fluxo principal

O checkout concentra a orquestração do Gateway. A sequência implementada é:

1. Gera x-pedido-id
2. Valida x-api-key
3. Aplica rate limit
4. Valida token na Identidade
5. Consulta Cardápio e calcula total
6. Prepara ingredientes e consulta Estoque
7. Verifica disponibilidade de estoque
8. Envia cobrança ao Pagamento
9. Pagamento aciona o Orquestrador
10. Gateway executa a baixa de Estoque
11. Logger é acionado de forma não bloqueante
12. Gateway responde ao cliente

13.1 Cálculo do pedido

O preço é calculado a partir do Cardápio, não a partir de valor informado pelo cliente. O Gateway valida existência, disponibilidade e quantidade de cada pizza.

13.2 Consumo de ingredientes

Os documentos disponíveis não definem a quantidade de cada ingrediente consumida por pizza. Para o fluxo implementado, foi adotada a hipótese técnica de 1 unidade de cada ingrediente listado na receita por pizza, multiplicada pela quantidade pedida. Essa hipótese deve ser apresentada como decisão de implementação, não como regra oficial do serviço de Estoque.

13.3 Teste final do checkout

Após a correção da serialização do token, o checkout passou pela Identidade e chegou ao Cardápio. No teste com Calabresa, o Cardápio informou a pizza como indisponível e o Gateway retornou corretamente HTTP 400.

```
{
  "error": "PIZZA_INDISPONIVEL",
  "message": "Pizza 'Calabresa' está indisponível."
}
```

Esse resultado é considerado correto: a condição real da pizza foi preservada e não foi alterada artificialmente para produzir um checkout 200.

14. GET /v1/metrics

Endpoint protegido por x-api-key que expõe informações operacionais do Gateway.

```

{
  "servico": "API Gateway",
  "versao": "v1",
  "status": "UP",
  "rate_limit": {"limite_requisicoes": 5, "janela_segundos": 60},
  "retry": {"max_tentativas": 3, "intervalo_ms": 1000},
  "integracoes": {
    "cardapio": true,
    "estoque": true,
    "pagamento": true,
    "identidade": true,
    "orquestrador": true
  },
  "fallback": {
    "status": "PENDENTE",
    "motivo": "URLs oficiais das versões B não fornecidas"
  }
}

```

15. Logger e observabilidade

O Logger foi conectado de forma não bloqueante para que indisponibilidade de observabilidade não impeça a resposta de negócio. Nos testes, as rotas externas de Logger consultadas retornaram 404/inativas. Por isso, a integração permanece configurada, mas a indisponibilidade externa deve ser declarada na apresentação.

O node de Logger do checkout possui timeout configurado para evitar que a chamada externa mantenha a execução presa.

16. Retry e fallback

Os fluxos de Cardápio e Estoque possuem retry configurado com até 3 tentativas totais e intervalo de 1000 ms. O contrato global prevê retry para 5xx/timeout e fallback A -> B.

Limitação conhecida: o mecanismo nativo Retry On Fail do n8n deve ser conferido quanto à restrição exclusiva a 5xx/timeout, pois um retry indiscriminado de 4xx não representa a regra desejada. Para Pagamento, retry cego foi evitado por risco de cobrança duplicada.

Fallback A -> B não foi fabricado onde a URL oficial da versão B não estava disponível. A documentação final registra essa dependência externa explicitamente.

17. Códigos HTTP tratados

Código	Uso no Gateway
200	Consultas e integrações bem-sucedidas
201	Criação de cliente e criação de pedido quando aplicável
400	Dados inválidos, pizza inexistente/indisponível, estoque

	insuficiente
401	x-api-key inválida ou token inválido
403	Cliente bloqueado/inativo no fluxo de Pagamento/Identidade
404	Recurso/rota não encontrada
422	Saldo insuficiente traduzido pelo Pagamento
429	Rate limit do Gateway excedido
500	Erro interno observado em serviços externos
503	Indisponibilidade/timeout após tratamento

18. Evidências de testes

Cenário	Resultado
Health do Gateway	Implementado
Menu com chave válida	200
Menu com chave inválida	401
Estoque - consulta válida	200
Estoque - chave inválida	401
Estoque - erro externo	500 observado
Pagamento válido	200 APROVADO
Pagamento valor inválido	400
Orquestrador - criação	201
Identidade - token válido direto	200
Checkout - token válido	Identidade passou após correção
Checkout - pizza indisponível	400 PIZZA_INDISPONIVEL
Rate limit - 6ª requisição	429
Metrics produção	200
Cadastro + Redis produção	201
Redis GET cliente salvo	Registro recuperado

19. Limitações e pendências externas

- URLs oficiais das versões B não estavam disponíveis em todos os documentos recebidos; o fallback não foi inventado.
- O endpoint externo de Logger permaneceu indisponível/não registrado durante os testes.
- O serviço de Estoque apresentou 500 para ingrediente inexistente em situação documentada como 400.
- A disponibilidade das pizzas depende do Cardápio externo e foi preservada sem alterações.
- O Redis Cloud utilizado na demonstração é auxiliar e não é a fonte autoritativa do Cadastro.
- A regra de 1 unidade de cada ingrediente por pizza é uma hipótese de implementação por ausência de quantidade de receita no contrato disponível.

20. Roteiro sugerido para demonstração

- Mostrar o workflow principal do API Gateway no n8n.
- Executar GET /health para provar que o serviço está online.

- Executar GET /v1/metrics com x-api-key válida.
- Mostrar POST /v1/clientes retornando 201 e a gravação auxiliar no Redis.
- Executar a validação de um token recém-criado.
- Executar POST /v1/checkout e explicar que a resposta 400 PIZZA_INDISPONIVEL é a aplicação correta da regra retornada pelo Cardápio.
- Mostrar o x-pedido-id sendo gerado e propagado pelos nodes do checkout.
- Explicar rate limit, retry, tratamento de erros e a pendência de fallback B/Logger como dependências externas.

21. Conclusão

O Serviço 01 - API Gateway do PZaaS foi implementado como ponto único de entrada e integração dos serviços do projeto. A solução possui autenticação por x-api-key, rastreabilidade por x-pedido-id, rate limit, tratamento de erros, métricas, integrações com os serviços principais e um fluxo de checkout que respeita as respostas reais dos serviços externos. As limitações dependentes de outras equipes foram mantidas explícitas, sem criação de URLs ou comportamentos não suportados pelos contratos recebidos.

Apêndice A - URLs principais

Finalidade	URL
Base produção	https://veagabronx2k.app.n8n.cloud/webhook
Health	https://veagabronx2k.app.n8n.cloud/webhook/health
Menu	https://veagabronx2k.app.n8n.cloud/webhook/v1/menu
Estoque	https://veagabronx2k.app.n8n.cloud/webhook/v1/estoque/consulta
Pagamento	https://veagabronx2k.app.n8n.cloud/webhook/v1/pagamento
Pedido	https://veagabronx2k.app.n8n.cloud/webhook/v1/pedido
Validar token	https://veagabronx2k.app.n8n.cloud/webhook/v1/auth/validar
Criar cliente	https://veagabronx2k.app.n8n.cloud/webhook/v1/clientes
Checkout	https://veagabronx2k.app.n8n.cloud/webhook/v1/checkout
Metrics	https://veagabronx2k.app.n8n.cloud/webhook/v1/metrics